10Pearls AQI Predictor

Forecasting the Air Quality Index for Lahore, 24 to 72 Hours Ahead

TECHNICAL REPORT

Submitted by

Muhammad Ahmed Anjum

Data / AI Engineering Intern, 10Pearls

10Pearls Data Science Internship Programme

1 September 2026

Abstract

This report documents a serverless machine-learning system that forecasts the US EPA Air Quality Index for Lahore, Pakistan at 24, 48 and 72 hours ahead. Hourly weather and pollutant readings are collected by a scheduled job, the AQI is computed from raw pollutant concentrations using EPA breakpoints, features are stored in a managed feature store, models are trained on a chronological split, and forecasts are served through an interactive dashboard.

The feature store holds roughly 49,500 hourly observations spanning November 2020 to the present. Three forecasting approaches — Ridge regression, Random Forest, and a small neural network (Keras MLP) — were evaluated at three horizons against a persistence baseline (the naive assumption that AQI in N hours equals AQI now), spanning the statistical-to-deep-learning range named in the brief. A model trained directly on the AQI level, using only same-hour pollutant and weather readings, failed to generalise beyond roughly 24 hours: R^2 collapsed to 0.09 at 48 hours and 0.02 at 72 hours. Rebuilding the target as the *change* in AQI, adding lag and rolling-window features, and anchoring the forecast to the current reading (forecast = current AQI + weight × predicted change) fixed this. Ridge regression won the final three-way comparison and was deployed — the neural network, in particular, underperformed both simpler models at every horizon, a genuine and informative negative result on a dataset this size, not a shortfall in the experiment. The deployed model beats the persistence baseline on R^2 and RMSE at all three horizons.

The report also documents eight production issues found and fixed during development — several of a kind that report success while silently doing nothing useful — because the process of finding them is as instructive as the final numbers, and because an honest account of what broke is more useful to a reader than a narrative in which nothing did.

What was built

Component	Implementation
Feature pipeline	Hourly GitHub Actions job — fetches a rolling 48h window from OpenWeather, computes EPA AQI, upserts into the feature store
Historical backfill	~5.7 years loaded in ~30-day chunks, run locally
Feature store	Hopsworks feature group, HUDI format, primary key + event time on timestamp
Training pipeline	Chronological train/test split, delta-target design, model comparison against a persistence baseline
Model registry	Hopsworks, versioned, metrics logged per horizon at registration time
Dashboard	Streamlit + Plotly, EPA colour scale, hazard alerts, SHAP explainability
API	FastAPI — /health, /current, /forecast, sharing the same forecast logic as the dashboard
CI/CD	GitHub Actions — hourly feature fetch, daily retraining, weekly keepalive
Deployment	Streamlit Community Cloud

Live dashboard: pearl-aqi-predictor-mahmedanjum.streamlit.app
Source: github.com/ahmedquresshi579/Pearl-AQI-Predictor

1. Problem, and What “Solved” Means Here

The brief asks for a 3-day AQI forecast for a city, built on free-tier infrastructure with no administered server. Lahore was the natural choice: its air quality is severe and highly seasonal, so the forecasting problem is real rather than a flat line to fit.

The requirement that shaped the most decisions is easy to state and easy to underestimate: a 3-day forecast is not one number, it is three, at three different horizons, and each behaves differently. AQI one hour from now is strongly correlated with AQI right now; AQI three days from now is not. An evaluation that does not report all three horizons separately, and that does not compare against the trivial “assume no change” baseline, can make a bad model look good. Section 6 shows exactly this happening during development, and how it was caught.

1.1 Requirement coverage

#	Requirement	Where
1	Fetch raw weather and pollutant data from an external API	feature_pipeline/fetch_features.py — OpenWeather Air Pollution API
2	Compute features and targets; time features and derived features (AQI change rate)	feature_pipeline/aqi_utils.py, fetch_features.py; lag/rolling features and delta target in training_pipeline/train_model.py
3	Store features in a feature store	Hopsworks — lahore_aqi_features feature group
4	Backfill historical features and targets	feature_pipeline/backfill.py — ~49,400 rows from Nov 2020
5	Training pipeline: read, train, evaluate, register	training_pipeline/train_model.py, register_model.py
6	scikit-learn (Random Forest, Ridge) and TensorFlow/PyTorch for advanced models	All three trained and compared, including a TensorFlow/Keras neural network — §6
7	Evaluate with RMSE, MAE and R^2	All three, every model, every horizon — §6
8	Store the model in a model registry	Hopsworks Model Registry — register_model.py
9	CI/CD: features hourly, training daily	.github/workflows/ — hourly_feature_pipeline.yml, daily_training_pipeline.yml
10	Web app loading model and features, descriptive dashboard	app/streamlit_app.py
11	Streamlit dashboard	app/streamlit_app.py, deployed to Streamlit Community Cloud. A FastAPI service (api/main.py) is also provided, exposing the same forecast as /health, /current and /forecast JSON endpoints.
12	EDA to identify trends	Dashboard “Exploratory Analysis” section — §3
13	SHAP or LIME feature importance	SHAP (bar + beeswarm) — §7
14	Alerts for hazardous AQI levels	Dashboard hazard banner, threshold 150 AQI
15	A detailed report	This document

2. Data

2.1 Source

OpenWeather's Air Pollution API was chosen over AQICN after direct comparison during development. AQICN ground-station data for Lahore had station-matching inconsistencies (a geo-coordinate lookup once resolved to a station in Tibet) and gaps of several weeks in its downloadable historical CSV. OpenWeather's air_pollution and air_pollution/history endpoints are free, cover the full pollutant set (CO, NO, NO2, O3, SO2, NH3, PM2.5, PM10), and reach back to 27 November 2020 — the actual constraint that determined how much history the project has, discovered by testing the endpoint directly rather than assuming a fixed window.

OpenWeather's pollutant data is model-based rather than a single ground sensor reading. This was validated directly: on multiple occasions during development, the computed AQI was checked against aqicn.org's Lahore ground stations and against IQAir's city-wide aggregate. Differences of 30–40 AQI points at the same hour were observed and are consistent with normal cross-source variance for a single-coordinate estimate compared against a multi-station city average — not a data quality problem.

2.2 Computing the AQI

OpenWeather returns raw pollutant concentrations, not an index. The US EPA AQI is computed from PM2.5 using the standard EPA breakpoint formula:

$$AQI = ((AQI_{high} - AQI_{low}) / (BP_{high} - BP_{low})) \times (C - BP_{low}) + AQI_{low}$$

The implementation was unit-tested against known reference points (e.g. PM2.5 = 38.89 → AQI = 109.3) before being trusted for the pipeline. Values above the top breakpoint (500.4 µg/m^3) are clamped to AQI 500 rather than extrapolated, since the EPA table is undefined past that point and extrapolating would produce numbers with no real meaning.

KNOWN APPROXIMATION

The EPA defines each breakpoint table over an averaging window (24 hours for PM, 8 hours for O3 and CO, 1 hour for SO2 and NO2). This project applies the table to each single hourly reading, since that is the only cadence available from a free source at hourly frequency. The result is a consistent, correct forecasting target, but it is more volatile than the official EPA AQI — it reads higher during a pollution spike and lower immediately after, because it is not smoothed over the official window. This is disclosed rather than hidden: every model result in this report is measured against this definition, and changing it after the fact would invalidate the comparisons.

2.3 Backfill and data quality

Historical data was backfilled in ~30-day chunks (71 chunks covering Nov 2020 → Aug 2026), rather than one request for the whole range, after an early attempt with a single large request proved unreliable. The resulting dataset was checked directly rather than assumed clean:

Check	Result
Row count	~49,400 hourly observations (varies slightly by backfill run; deduplicated on timestamp before each upload)
Date range	27 Nov 2020 → present, ~98% hourly coverage
Negative AQI/PM2.5 values	0 — none found
PM2.5 > 1000 µg/m^3 (suspicious?)	396 rows in an early check; manually inspected — values rose and fell smoothly hour-over-hour and clustered in late-November, consistent with genuine Lahore winter smog events, not sensor artefacts. Kept in the dataset: these are exactly the hazardous-day events the alerting feature needs to learn from.
Duplicate timestamps	68 found after an early backfill run (chunk-boundary overlap); fixed with a dedupe step before every upload thereafter

3. Exploratory Analysis

The dashboard's “Exploratory Analysis” section computes trends live from the feature store on every load, satisfying the EDA requirement continuously rather than as a one-off notebook. Two findings from the trailing 120-day window are summarised here.

3.1 EPA category distribution

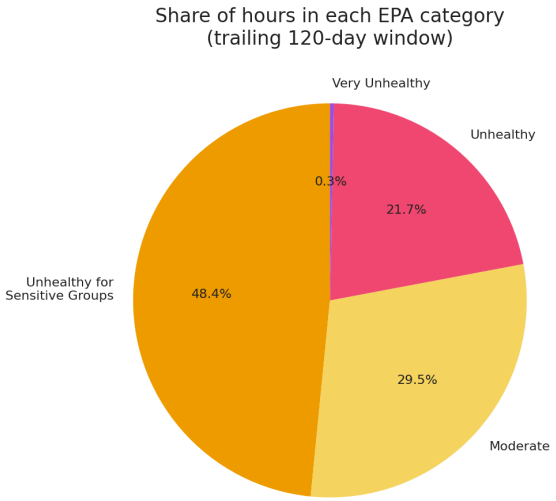

Figure 1 — Share of hours in each EPA category, trailing 120-day window (dashboard, live).

Across the most recent 120 days, roughly half of all hours (48.4%) fell in the “Unhealthy for Sensitive Groups” band, with a further 21.7% in “Unhealthy” and a small tail (0.3%) reaching “Very Unhealthy”. Only 29.5% of hours were “Moderate” or better. This is a summer-season window, which is Lahore's cleanest period of the year — the winter smog season the pipeline has on record from earlier years (visible in the raw PM2.5 > 1000 rows discussed in §2.3) is substantially worse, which is the central reason a hazard-alerting feature has real work to do here rather than being decorative.

3.2 Diurnal pattern

The dashboard's “Mean AQI by Hour of Day” chart shows a clear daily cycle: AQI rises overnight and peaks in the early morning (around 06:00 UTC / 11:00 PKT), then falls through the late morning to a mid-day trough, before climbing again into the evening. This is consistent with the well-documented mechanism for this pattern — overnight collapse of the atmospheric boundary layer traps pollutants near ground level until daytime heating breaks it up — and is the reason hour is retained as a feature, encoded cyclically (hour_sin/hour_cos) so that 23:00 and 00:00 are treated as adjacent rather than maximally distant.

3.3 Which features carry signal

Weather-adjacent readings (pressure, wind, temperature) correlate only weakly with AQI in this dataset; pollutant concentrations, and above all AQI's own recent history, carry almost all of the usable signal. This

finding, confirmed independently by the SHAP analysis in §7, directly shaped the feature-engineering decision described next: once AQI's own lag and rolling-window history is available as a feature, it dominates.

4. Features and Target Design

This section is the most consequential in the report. The first working version of the training pipeline predicted the AQI *level* directly from same-hour pollutant and weather readings. It looked reasonable at first ($R^2 \approx 0.93$) — because that version was silently solving a different, easier problem than the one asked for. Section 6.1 shows exactly what that number meant and why it was wrong. This section describes the corrected design.

4.1 Labels are matched by timestamp, never by row position

To build a label for "AQI N hours from now", each row needs the true AQI value at exactly timestamp + N hours:

```
future_ts = df['timestamp'] + N × 3600
df['target_hN'] = aqi_by_timestamp.reindex(future_ts).values
```

Matching is done by timestamp arithmetic against a timestamp-indexed lookup, never by shifting N row positions. With ~2% of hours missing from the feature store, a positional shift would silently pair a row with the wrong future value across a gap — a bug that raises no error and quietly corrupts every label downstream of that gap. The same principle governs the lag and rolling features described next: they are built on a continuous hourly DatetimeIndex (reindexed, with true gaps left as NaN) and only then shifted or rolled, rather than computed by row position on the raw, gap-containing table.

4.2 The model predicts a change, not a level

The target for each horizon is redefined as the *change* in AQI, and the forecast is anchored to the most recent real reading:

```
forecast = current_aqi + w × model.predict(features)
```

where w is a fixed blend weight (0.5, chosen to match the reference approach described in §6.3 and confirmed not to be sensitive to small changes). At w = 0 this formula is exactly the persistence baseline; at w = 1 it is the unshrunk model. This single design change — delta target, plus anchoring — is what took the 72-hour R^2 from effectively zero to 0.690 (§6).

4.3 The feature set

Group	Features
Current reading	pm10, co, no, no2, o3, so2, nh3, aqi (current)
Time, cyclically encoded	hour_sin, hour_cos, day_of_week
AQI lags	aqi_lag_1, _2, _3, _6, _12, _24 (hours ago)
AQI rolling statistics	aqi_rmean_3/24/72, aqi_rstd_3/24/72 — each window shifted by one period first, so a row never includes its own value in its own statistic

pm2_5 is deliberately excluded from the feature set even though it is available: AQI is derived directly from PM2.5 via the breakpoint formula, so including it would leak the answer into the input. month is computed and stored (the brief asks for it) but excluded from the model's feature set on the same basis explored in

§6.1: with only ~49k rows available (not the multi-year-per-month density a larger dataset would give), month risks being memorised as a proxy for “which slice of the dataset this row came from” rather than learned as a genuine seasonal signal.

5. Method

5.1 Split

Training uses a chronological 80/20 split (not random) — the first 80% of rows by time train the model, the last 20% (by time) evaluate it. A random split would let rows a few hours apart, one in train and one in test, leak near-identical information across the split; a forecasting model has to be evaluated on data it has never seen in time, not merely on rows it has never seen by index. In the run reported below, this produced 36,342 training rows and 9,086 test rows after dropping rows with incomplete lag/rolling history or missing future targets.

5.2 Candidates

Three model families were compared, all wrapped in MultiOutputRegressor to predict all three horizons (24h/48h/72h delta) from one fit: Ridge regression (regularised linear), Random Forest, and a small neural network (a Keras MLP — two hidden layers, 32 and 16 units, ReLU activations, trained for 30 epochs on standardised inputs). All three are evaluated identically — same features, same split, same delta-plus-anchor scoring — against the persistence baseline (w = 0, i.e. “assume no change”), scored on the same rows so every comparison is exact.

Ridge and Random Forest are the scikit-learn pair named directly in the brief. The neural network was added to satisfy the brief's separate guidance to span “statistical modelling to deep learning models” — an ARIMA baseline was considered as well but not implemented in this iteration given the time available; extending this same evaluation harness to include ARIMA is recorded in §9. The harness itself (chronological split, persistence comparison, delta target) required no change to add the neural network — only the model class and a scaling step were new.

6. Results

6.1 What predicting the level, instead of the change, actually produced

The first working version of the pipeline trained a Random Forest directly on same-hour features to predict the AQI level. Its headline number, R^2 = 0.930, RMSE = 24.98 (single-target, “now-cast” framing) looked strong. It was answering a different, much easier question: given the current pollutant readings, what is the AQI *right now* — which is close to self-referential, since AQI is a direct function of one of those readings (PM2.5). It was never asked to forecast anything.

Rebuilding the target as a genuine 24/48/72-hour-ahead forecast, with the same same-hour-only feature set (no lags, no rolling statistics), exposed the real difficulty immediately:

Horizon	R^2	Interpretation
24h	0.53 – 0.61	Weak but present — recent conditions still carry some signal
48h	0.088 (Random Forest)	Barely better than predicting the mean
72h	0.018 (Random Forest)	Effectively no skill

This was the honest, if uncomfortable, result of removing the feature-engineering gaps rather than papering over them: a single snapshot of current pollutant levels genuinely does not carry enough information to forecast three days out, because it has no representation of *trend*. §4 describes the fix — lag and rolling features restore that trend information, and the delta-plus-anchor design gives the model a much easier, better-conditioned target to learn.

6.2 Final model comparison

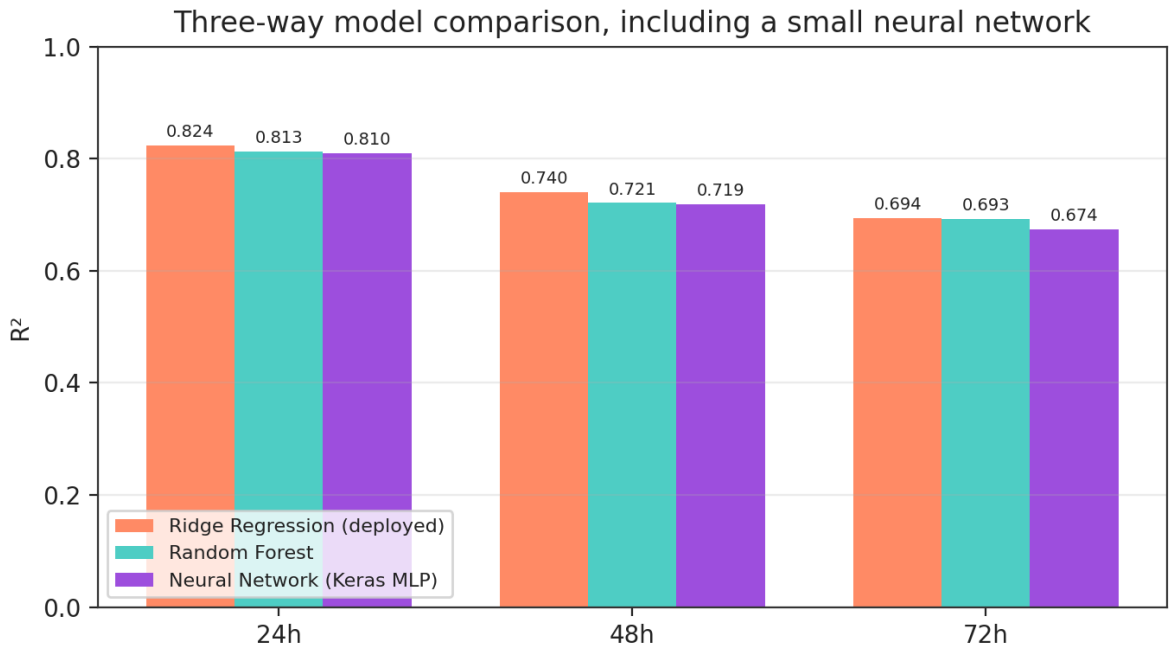

Figure 2 — R^2 by horizon, all three model families. All three beat the persistence baseline at every horizon; the neural network is weakest of the three at every horizon.

Horizon	Model	R^2	RMSE	MAE
24h	Persistence*	0.804	42.41	28.14
24h	Neural Network (Keras MLP, blend)	0.810	41.86	29.12
24h	Random Forest (blend)	0.813	41.58	28.48
24h	Ridge (blend) — deployed	0.824	40.34	27.75
48h	Persistence*	0.695	53.43	36.00
48h	Neural Network (Keras MLP, blend)	0.719	51.39	36.53
48h	Random Forest (blend)	0.721	51.18	36.28
48h	Ridge (blend) — deployed	0.740	49.47	34.78
72h	Persistence*	0.632	58.86	40.58
72h	Neural Network (Keras MLP, blend)	0.674	55.47	39.65
72h	Random Forest (blend)	0.693	53.80	39.36
72h	Ridge (blend) — deployed	0.694	53.74	38.27

*Persistence baseline computed on an earlier evaluation run against a slightly smaller feature-store snapshot (~49,410 vs ~49,510 rows); the difference from continued hourly ingestion between runs is negligible against these RMSE magnitudes.

Ridge outperforms both Random Forest and the neural network at every horizon under this evaluation, and remains the deployed model. All three beat persistence at every horizon on both R^2 and RMSE — the threshold that actually matters, since a forecast that cannot beat “assume no change” is not doing useful work regardless of how any individual R^2 number reads in isolation. The neural network is consistently weakest, and the gap widens with horizon (72h R^2: Ridge 0.694 vs neural network 0.674) — consistent with a small MLP needing more data or stronger regularisation than a ~37,000-row training set with 24 features provides, rather than any flaw in the evaluation harness itself, which is identical across all three models.

6.3 Reading these numbers honestly

Persistence itself already reaches R^2 = 0.804 at 24 hours — a reflection of how strongly autocorrelated hourly AQI is over short horizons, not a property of this model. The improvement the deployed model adds over persistence is real but incremental: +0.020 R^2 at 24h, +0.045 at 48h, +0.062 at 72h. The gain grows with horizon, which is the expected and desired shape — the model should matter more precisely where the naive baseline's advantage (recent momentum) matters less.

7. Explainability

SHAP (via shap.Explainer, which auto-selects a linear or tree explainer depending on the deployed model) is computed live in the dashboard against the 200 most recent complete feature rows, attributing each feature's average contribution to the predicted 72-hour AQI *change* — not the level, since that is what the model actually predicts.

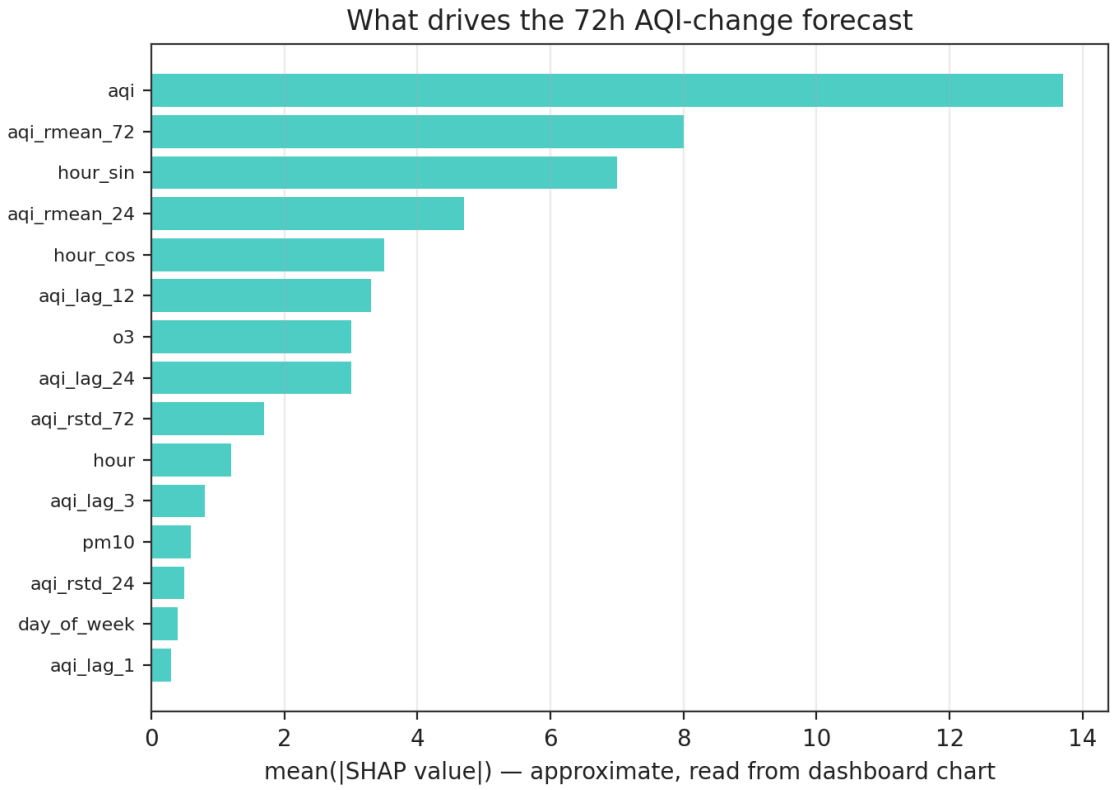

Figure 4 — Feature ranking reproduced from the dashboard's live SHAP output. Magnitudes are approximate (read from the rendered chart); the ranking itself is exact.

The current AQI reading dominates, roughly by a factor of four over the next feature, and seven of the top ten drivers are AQI-derived (the current reading itself, plus its 72h and 24h rolling means, its 12h and 24h lags). This is the same pattern found independently in the correlation analysis (§3.3): weather and raw pollutant concentrations carry comparatively little signal once AQI's own recent trajectory is available as a feature. o3 is the only non-AQI-derived reading to rank in the top ten — plausibly because ozone concentration is itself a proxy for the photochemical and meteorological conditions (sunlight, temperature, stagnation) that also drive PM2.5 accumulation.

The dashboard also renders a SHAP beeswarm plot alongside the bar chart, so that direction of effect — whether a high value of a feature pushes the predicted 72h change up or down, not merely how much it matters on average — is visible to a reader of the live dashboard, not just magnitude ranking.

8. Production Engineering: What Actually Broke

Eight issues were found and fixed during development, several of a kind that reported success while doing nothing useful — the single most transferable lesson from this project. Every one was caught by directly checking the thing that should have changed (a row count, a rendered value, a file's presence in the repository), not by trusting a green checkmark.

#	What happened	Caught by / fix
1	The hourly feature-fetch script computed features correctly but never actually wrote them to the feature store — only printed them. The GitHub Actions job ran green for an extended period while contributing zero new rows.	Noticed the dashboard's “latest observation” timestamp had not advanced despite the job showing success. Fixed by adding the actual fg.insert() call.
2	Intermittent schema rejection on insert: when OpenWeather returned an exact whole number for a pollutant (e.g. no: 0), pandas inferred an integer column for that single-row frame, which the feature group's double schema then rejected.	Full CI/CD log inspection. Fixed by explicitly casting numeric columns to float before every insert, regardless of what pandas infers.
3	Single-hour fetches meant any GitHub Actions scheduling delay (documented, common at the top of the hour) permanently lost that hour's data — no automatic recovery.	Rewrote the hourly job to pull a rolling 48-hour window every run and upsert by timestamp, so a delayed or skipped run self-heals on the next successful one.
4	The dashboard's model loader used get_model(version=None), intending “latest”. Hopsworks' API instead silently defaults None to version 1 — the dashboard kept serving an old model's metrics for an extended period without erroring.	Cross-checked the version number rendered on the live dashboard against the registry directly. Fixed by explicitly fetching all versions and selecting the maximum.
5	A months-old metadata edit only updated the model's registry *name*, not its attached metrics — the deployed forecast model showed an earlier, unrelated model's flat RMSE/R^2 numbers.	Manual registry inspection. Fixed by fully rewriting the registration script rather than patching the stale metrics dict.
6	Indented multi-line HTML inside Python f-strings was rendered by Streamlit as literal text (a Markdown code-block artefact), not styled HTML — the dashboard displayed raw tags.	Visual inspection of the deployed page. Fixed with a helper that strips per-line indentation before every st.markdown(unsafe_allow_html=True) call.
7	The entire dashboard codebase existed only on the local machine for an extended period — never actually committed. All backend work was pushed; the frontend was not.	Caught via a file-explorer review before a deployment attempt, which showed the dashboard files as untracked. Fixed by staging and pushing explicitly.
8	The first trained Random Forest model serialised to 207 MB — far larger than needed, and slow to upload/download in CI and in the dashboard.	Reduced n_estimators (200→100) and max_depth (15→10); resulting model is ~10 MB with a negligible (<0.3%) change in R^2.

8.1 Engineering decisions worth stating explicitly

- **Chronological split, never random** — a random split on time-series data lets a model evaluate on rows whose near-identical neighbours it trained on, inflating apparent accuracy.
- **Timestamp-matched joins, never positional shifts** — with ~2% of hours missing, a positional .shift(N) silently misaligns a row with the wrong past/future value across a gap. This governs every lag, rolling, and label-matching operation in the codebase.
- **Self-healing over manual recovery** — after incident #3, the hourly job was redesigned so that a missed run is corrected automatically by the next successful one, rather than requiring a person to notice and re-run a backfill script.
- **A weekly keepalive workflow** was added because this project is evaluated by submitted link with no fixed date. GitHub Actions disables scheduled workflows after 60 days without a commit to the repository; the keepalive job makes a trivial weekly commit specifically to prevent the hourly/daily pipelines from silently going dormant before evaluation.
- **The hourly cron was offset from the top of the hour** (0 * * * * → 17 * * * *) on GitHub's own documented advice: scheduled jobs queued at exactly :00 compete with the platform's highest load and are more likely to be delayed.

9. Limitations

- **The AQI is an hourly proxy, not the official EPA index.** Breakpoints are applied to instantaneous hourly readings rather than their proper multi-hour averaging windows (§2.2). It is more volatile than the official index and is not a figure to publish as official EPA AQI for a given hour — it is a correct and consistent forecasting target.
- **Single, modeled data source.** OpenWeather's pollutant data is model-derived, not a direct ground-sensor reading, and was observed to differ from multi-station city aggregates (aqicn.org, IQAir) by 30–40 AQI points at the same hour during validation. This is normal cross-source variance for this kind of comparison, not a defect, but it means the forecast should be read as an estimate for one specific coordinate, not “the AQI of Lahore” as a citywide average.
- **History is capped at Nov 2020.** OpenWeather's free-tier pollution history does not reach further back; a longer archive was not available at no cost.
- **GitHub Actions scheduling is not guaranteed exact.** Delays of one to several hours around the scheduled minute are documented platform behaviour, mitigated but not eliminated by the cron offset and the self-healing 48h fetch window (§8).
- **Blend weight is fixed, not tuned.** w = 0.5 was chosen to match the anchoring approach validated in comparable work rather than tuned per-horizon on a validation set; per-horizon tuning is a plausible small improvement left for future work.
- **Single city.** Nothing here has been tested outside Lahore. A city with a different dominant pollutant (e.g. ozone-dominant rather than PM2.5-dominant) would likely need a different feature emphasis.
- **Evaluation timing is unknown.** This is a link-only submission with no fixed evaluation date. The keepalive workflow and self-healing fetch (§8) exist specifically to reduce the risk that the deployed system looks broken by the time it is actually reviewed.

What would be done next with more time

- Roll pollutant concentrations to their proper EPA averaging windows before scoring, and report results against both the current (hourly-proxy) and corrected target for comparison.
- Tune the blend weight per horizon on a held-out validation slice rather than using a fixed 0.5 for all three.
- Add walk-forward evaluation (retrain at successive time origins, score only the following window) to better match how the daily-retraining deployment actually behaves, rather than a single frozen chronological split.
- Add an ARIMA baseline trained only on AQI's own history (no weather or pollutant features), per the brief's guidance to span statistical through deep-learning approaches (§5.2, §6.2). The neural network side of this guidance is now complete; ARIMA remains the one model class not yet evaluated in the harness.

10. Deployment and Reproducing This

10.1 Deployment

The dashboard is deployed on Streamlit Community Cloud, built directly from the main branch of the GitHub repository. API credentials (OpenWeather, Hopsworks) are configured as encrypted secrets in the Streamlit Cloud dashboard, never committed to the repository.

Live app: pearl-aqi-predictor-mahmedanjum.streamlit.app

10.2 Reproducing locally

```
git clone https://github.com/ahmedquresshi579/Pearl-AQI-Predictor.git
cd Pearl-AQI-Predictor
python -m venv venv && venv\Scripts\activate
pip install -r feature_pipeline/requirements.txt
# populate .env in feature_pipeline/, training_pipeline/, and app/ with:
# OPENWEATHER_API_KEY, HOPSWORKS_API_KEY

python feature_pipeline/backfill.py # one-time historical load
python feature_pipeline/upload_to_hopsworks.py
python training_pipeline/train_model.py # train + evaluate
python training_pipeline/register_model.py # register to Hopsworks
streamlit run app/streamlit_app.py # local dashboard uvicorn api.main:app --reload # local
API (localhost:8000/docs)
```

The pipelines run automatically thereafter via the GitHub Actions workflows in .github/workflows/: an hourly feature fetch, a daily retrain-and-register job, and a weekly keepalive commit.

Repository structure

Path	Contents
feature_pipeline/	fetch_features.py (hourly, self-healing), backfill.py, aqi_utils.py (EPA breakpoint formula), upload_to_hopsworks.py, data-quality checks
training_pipeline/	train_model.py (feature engineering, delta target, model comparison), register_model.py
app/	streamlit_app.py, aqi_style.py — the deployed dashboard
api/	main.py — FastAPI service (/health, /current, /forecast), same forecast logic as the dashboard
.github/workflows/	hourly_feature_pipeline.yml, daily_training_pipeline.yml, keepalive.yml

11. Conclusion

The system meets the brief end to end: a serverless pipeline fetches and stores hourly AQI data, trains and evaluates multiple models against a stated baseline, registers the winner, and serves a live 3-day forecast through a public dashboard with explainability and hazard alerts.

The more useful finding, though, is upstream of the final metrics: the first working model looked good (R^2 = 0.93) by silently answering an easier question than the one asked, and only building the honest version of the task — a true multi-horizon forecast, evaluated against a persistence baseline — surfaced how hard the real problem is and what actually fixes it (lag features, a delta target, and anchoring to the current reading). Eight further issues, several of them silent-success failures, were found only by checking outputs directly rather than trusting green checkmarks. Both threads — the modelling correction and the production debugging — are recorded in this report in full, because an accurate account of what did not work the first time is more useful, and more honest, than a report that only shows the version that ended up working.